

STM32 Traffic Light System Project

February - 2025

Table of Contents

Table of Contents	2
List of Figures and Tables:	2
1. Introduction	3
2. Discussions and Procedure	4
2.1 System Overview	4
2.2 Middleware	5
2.1.1 GPIO Pins	5
2.1.2 Implementation Details	5
2.3 Software	6
2.3.1 Traffic Generating Algorithm	11
2.3.2 System Display Algorithm	11
3. Conclusion	12
4. References	13

List of Figures and Tables:

<u>Figure 1: Flowchart of overall system.....</u>	<u>4</u>
<u>Figure 2: Code of GPIO.....</u>	<u>6</u>
<u>Figure 3: Code of ADC.....</u>	<u>6</u>
<u>Figure 4: Code of Traffic Flow Adjustment Task.....</u>	<u>7</u>
<u>Figure 5: Code of Traffic Generator Task.....</u>	<u>8</u>
<u>Figure 6: Code of Traffic Light State Task.....</u>	<u>9</u>
<u>Figure 7: Code of System Display Task (1).....</u>	<u>10</u>
<u>Figure 8: Code of System Display Task (2).....</u>	<u>10</u>
<u>Figure 9: Flowchart of Traffic Generator Task.....</u>	<u>11</u>
<u>Figure 10: Flowchart of System Display Task.....</u>	<u>11</u>

1. Introduction

The Traffic Light System (TLS) project is designed to simulate a one lane road controlled by a traffic light. The primary objective is to implement a real-time embedded system using FreeRTOS features such as tasks, queues, and timers. The TLS dynamically adjusts traffic flow based on a potentiometer input, manages vehicle movement using LEDs, and controls the timing of traffic lights. This project provides hands-on experience with FreeRTOS features.

2. Discussions and Procedure

This section of the report will go over our procedure in great depth, explaining our methodologies and strategies in creating our project. It has been separated into three main sections, each outlining different aspects of our project.

2.1 System Overview

Figure 1 Below showcases the general workings of our program. Keep in mind the tasks are independently running continuously alongside the main repeating loop shown below. One full loop corresponds to about roughly one second and represents a singular tick in our system.

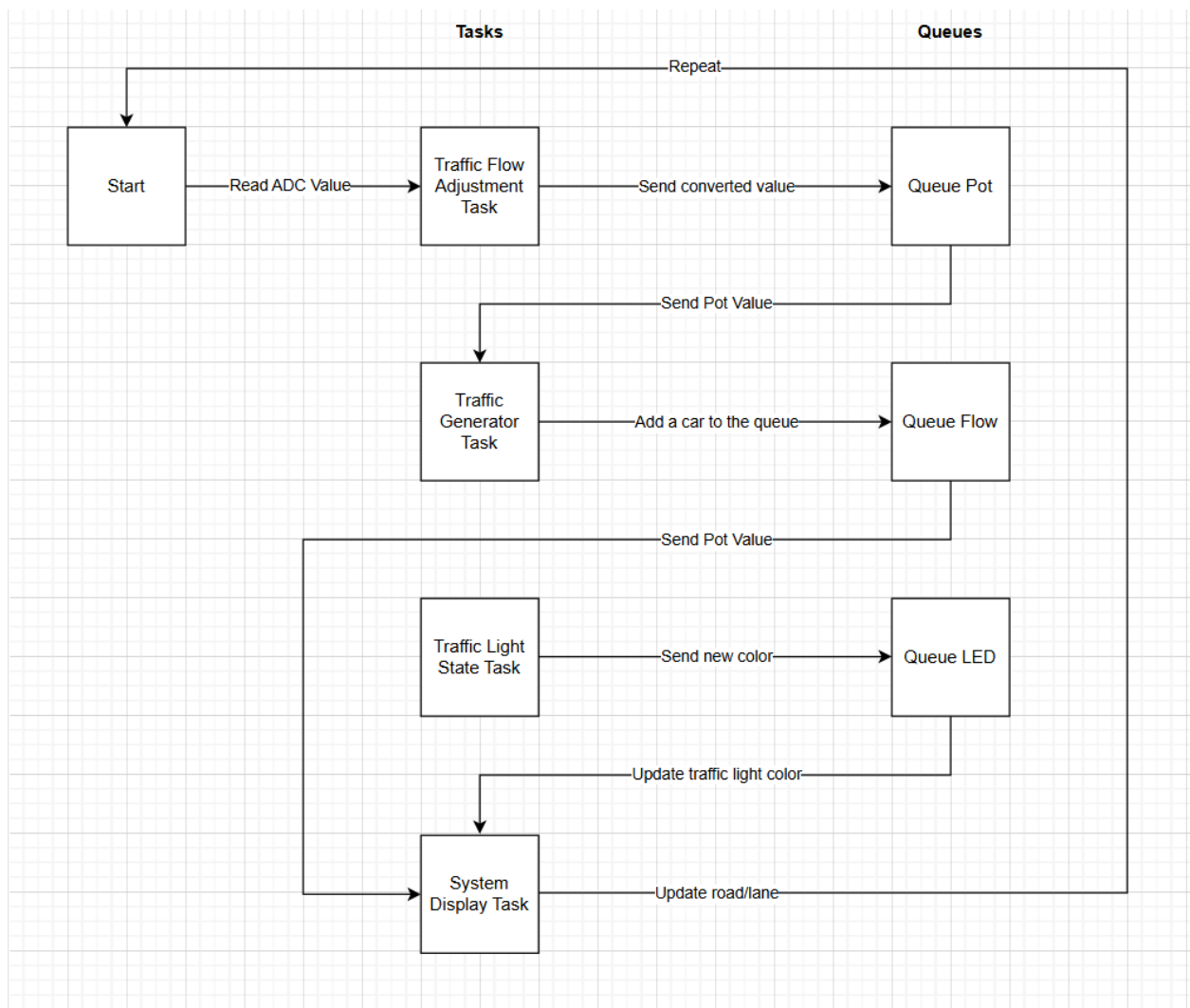


Figure 1: Flowchart of overall system

2.2 Middleware

2.1.1 GPIO Pins

Below is the list of GPIO pins utilized during this project followed by a short description outlining its use and purpose:

- Pin 8 (PC0): This pin is tied to the red light signal in our traffic light system. If on, no cars are allowed to pass the intersection.
- Pin 9 (PC1): This pin is tied to the amber light signal in our traffic light system. This is functionally the same as the red light; no cars are allowed to pass the intersection if this is on. The only difference is that the timer remains constant at all times.
- Pin 10 (PC2): This pin is tied to the green light signal in our traffic light system. If on, cars are permitted to pass the intersection and the whole lane is open.
- Pin 11 (PC3): This is connected to our potentiometer and its variable value was read by using the ADC. A high resistance value here means the traffic intensity is high. On the other hand, a small resistance value makes the flow of traffic low.
- Pin 37 (PC6): This is tied to the shift registers Data pin. This controls relevant data with 1's to represent a car and 0's representing the absence of a car in any particular spot on the road.
- Pin 38 (PC7): This is tied to the shift registers Clock pin. This controls the clock of the shift register to dictate how data is transferred along.
- Pin 39 (PC8): This is tied to the shift registers Reset pin. This is used to clear all of the data that is currently being held.

2.1.2 Implementation Details

GPIO: To begin utilizing the GPIO pins, we first had to initialize them within our program. This began by enabling the GPIO clock and creating a struct. This struct contains five fields which set different properties of the GPIO. PC3 (potentiometer) was set by itself due to requiring different properties while the rest were all configured in the same way. Once we established our struct fields in the manner desired, invoking `GPIO_Init` did the remaining work for us.

```

// configure GPIOC
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE); // enable the GPIOC clock
GPIO_InitTypeDef GPIO_struct;

GPIO_struct.GPIO_Pin = red | amber | green | data | clock | reset; // traffic light pins and shift registers
GPIO_struct.GPIO_Mode = GPIO_Mode_OUT; // set output mode
GPIO_struct.GPIO_Speed = GPIO_Speed_50MHz; // set speed
GPIO_struct.GPIO_OType = GPIO_OType_PP; // set push-pull mode
GPIO_struct.GPIO_PuPd = GPIO_PuPd_NOPULL; // disable pull-up / pull-down
GPIO_Init(GPIOC, &GPIO_struct);

GPIO_struct.GPIO_Pin = pot; // PC3 pin for POT
GPIO_struct.GPIO_Mode = GPIO_Mode_AN; // set analog mode
GPIO_struct.GPIO_PuPd = GPIO_PuPd_NOPULL; // disable pull-up / pull-down
GPIO_Init(GPIOC, &GPIO_struct);

```

Figure 2: Code of GPIO

ADC: To get our potentiometer working and allowing for there to be a line of communication between the program, the board and the user, we had to ensure the ADC was working. Just like with the GPIO, we begin by enabling the clock and creating a struct. In this case, the default settings for the fields of the struct were appropriate so it could be initialized immediately. The only difference is that the ADC sample time had to be modified. Through testing and experiments, we found that “144cycles” worked for us and yielded desired results.

```

// configure ADC
RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE); // enable the ADC clock

ADC_InitTypeDef pot_ADC_struct;
ADC_StructInit(&pot_ADC_struct);

ADC_Init(ADC1, &pot_ADC_struct);

ADC_Cmd(ADC1, ENABLE); // enable ADC1
ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_144Cycles); // ADC config

```

Figure 3: Code of ADC

2.3 Software

The program operates through the use of four tasks to generate a dynamic traffic simulation based on user input. The first task is “Traffic_Flow_Adjustment_Task” which involves adjusting the flow of traffic by reading the potentiometer value through the ADC. It also scales the

resistance value read from 0 to 100 to make future calculations more clean. This task is very simple and essentially acts as a bridge to better compartmentalize our code.

```
static void Traffic_Flow_Adjustment_Task( void *pvParameters )
{
    int adc_value;
    int pot_value;
    while(1)
    {
        adc_value = get_ADC_val();
        pot_value = (int)100*(adc_value-1600)/(4095-1600);

        if (xQueueOverwrite(xQueue_pot, &pot_value)){
            vTaskDelay(500);
        }
    }
}
```

Figure 4: Code of Traffic Flow Adjustment Task

Speaking of which, “Traffic_Generator_Task” is the task that receives the potentiometer value via a queue from our previous task. This is used to dictate how much of a delay there is between sending the next car into the traffic queue. Since the while loop constantly sends a card into the flow queue, changing the delay is enough to control how fast cars are being outputted. For this, we created an equation to model a linear slope that has the desired values at our relevant min and max input value, 0 and 100 respectively. A higher resistive value means that the delay is shorter and cars are sent into the queue much faster. Conversely, a low resistive value means that cars are being sent into the queue at a much slower rate. See section 2.3.1 for a flowchart which visualizes our code in a simplified manner.

```

static void Traffic_Generator_Task( void *pvParameters )
{
    uint16_t pot_val;
    int flow;

    while(1)
    {
        if(xQueuePeek(xQueue_pot, &pot_val, 1000))
        {
            if (xQueueOverwrite(xQueue_flow, &flow))
            {
                vTaskDelay(7000-(66*(pot_val)+1));
            }
        }
    }
}

```

Figure 5: Code of Traffic Generator Task

The next task is “Traffic_Light_State_Task” and is responsible for adjusting the time of the traffic lights based on the potentiometer value read. This section is a bit more complicated as it has the task itself but also feeds into a sort of loop by triggering timer handles for each light color. First, the green timer is triggered, turning off the green light and turning on the amber light. This puts us in the amber timer handle and once the amber light is turned off and the red is turned on, then triggering the red timer. Finally, the red timer being triggered indicates that the red light should end and we have reached the end of the loop. The task is now able to resume before triggering the whole time handle loop again. This effectively allows us to check the timers for each light using queues for real time operation. Furthermore, the green and red lights are inversely related based on the potentiometer while the amber light is always constant.


```

static void Traffic_Light_State_Task( void *pvParameters )
{
    int led_val = GREEN;

    printf("green on\n");
    green_control(1);
    xTimerStart(green_light_timer, 0);

    if (xQueueOverwrite(xQueue_led, &led_val)){
        vTaskDelay(500);
    }

    while(1){
        vTaskDelay(1000);
    }
}

```

Figure 6: Code of Traffic Light State Task

Finally, “System_Display_Task.” is responsible for outputting the data that our code runs. The first for loop utilizes the shift register to move each car along by one. Essentially, if a car is present at a certain location, it is moved forward by one space. Next, we receive the current led color so that we can decide if cars should be allowed to pass the stoplight. If we receive green, the cars can move freely along the entire road. Otherwise, cars behind the stop light must stop while those ahead can continue onwards. Finally, before the loop ends, we check the traffic flow to see if there are any cars that are waiting to enter the road from the queue. See section 2.3.2 for a flowchart which also visualizes our code in a simplified manner.

```

static void System_Display_Task( void *pvParameters )
{
    int led_val = GREEN;
    int flow;
    int cars[20] = {0};
    while(1)
    {
        // simulate cars moving
        for (int i = 19; i >= 0; i--){
            if (cars[i] == 1){
                shift_reg(1);
            }else{
                shift_reg(0);
            }
        }

        // get led value
        if (xQueueReceive(xQueue_led, &led_val, 500)){
            vTaskDelay(500);
        }

        // green light
        if (led_val == GREEN){
            for (int i = 19; i > 0; i--){
                cars[i] = cars[i-1];
            }
        }

        // amber and red light
        else if (led_val == AMBER || led_val == RED){
            // shifting cars before red light
            for (int i = 7; i > 0; i--){
                if (cars[i] == 0){ // if no car in the lane
                    cars[i] = cars[i-1]; // shift cars
                    cars[i-1] = 0; // clear the previous car
                }
            }

            // shifting cars after red light
            for (int i = 19; i > 8; i--){
                cars[i] = cars[i-1]; // shift cars after red light
                cars[i-1] = 0; // clear the previous car
            }
        }

        if (xQueueReceive(xQueue_flow, &flow, 100)){
            cars[0] = 1; // add car to the lane
        }else{
            cars[0] = 0; // add nothing to the lane
        }

        vTaskDelay(500);
    }
}

```

Figure 7 & 8: Code of System Display Task

The combination of these four tasks along with several other smaller functions in our program allow the simulation of a one lane traffic road that can be modified by users in real time. Beyond our tasks, most of the program consists of initializing and setting up pins so that our board and code can communicate effectively. Here and there, we included helper functions like “get_ADC_val” and “red_control” but those are purely to better the form and style of our code.

The sections below contain Figure 7 and Figure 8. This helps visualize what our key tasks, Traffic Generating and System Display, are doing in a step by step manner. It essentially captures the methodologies explained in the above paragraphs, outlining how the code operates. The splitting branches also explain when different scenarios arise and how the differences affect the program.

2.3.1 Traffic Generating Algorithm

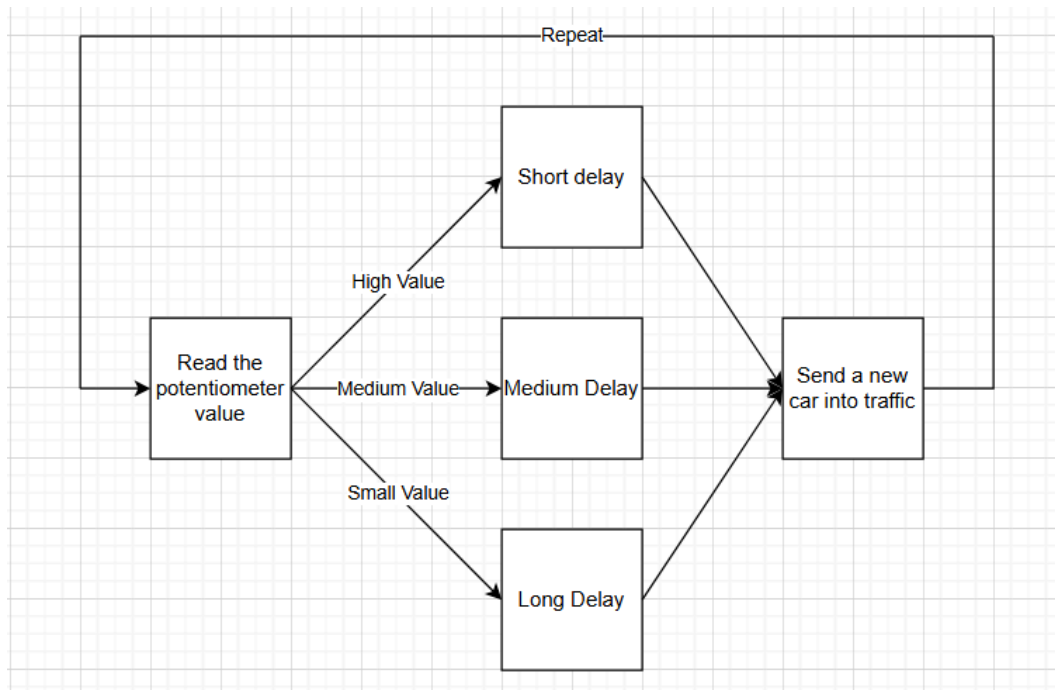


Figure 9: Flowchart of Traffic Generator Task

2.3.2 System Display Algorithm

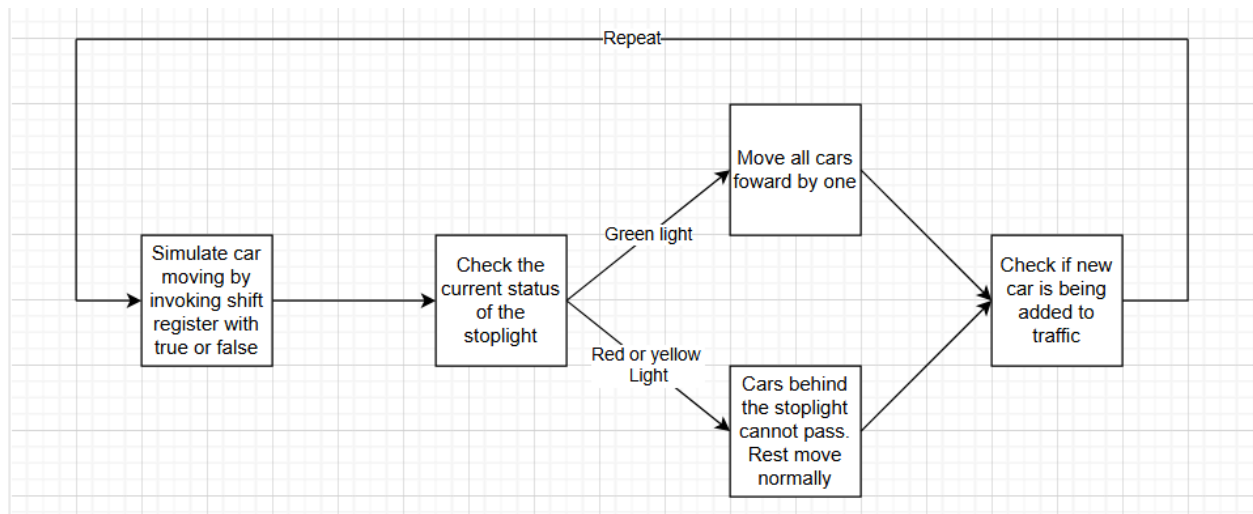


Figure 10: Flowchart of System Display Task

3. Conclusion

The implementation of the Traffic Light System successfully demonstrates real-time embedded system principles using FreeRTOS. By integrating hardware components with different software tasks, the system effectively manages vehicle flow and adapts to varying traffic conditions dynamically. The use of queues for task communication and timers for light control ensures scalability. Overall, this project provides valuable insights into real-time system development and embedded programming.

4. References

- [1] A. Jooya, S. Khoshbakht, N. Agarwal, S. Campbell, N.J. Dimopoulos, et al.
ECE 455:REAL TIME COMPUTER SYSTEMS DESIGN PROJECT, (2019). Accessed
Feb 26, 2025. [Online].